```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import classification_report, confusion_matrix
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras. layers import Dense
from tensorflow.keras.utils import to_categorical
```

```
from google.colab import drive
drive.mount('/content/drive')
file_path = '/content/drive/My Drive/datasets/creditcard.csv'
```

```
Mounted at /content/drive
```

```
df = pd.read_csv('/content/creditcard.csv')

# Drop any rows with NaN values to ensure a clean split and training
df = df.dropna()

X = df.drop("Class", axis=1).values
y = df["Class"].values

# Standardize features (Amount & PCA components)
scaler = StandardScaler()
X = scaler.fit_transform(X)

# Train-test split (stratify to maintain fraud ratio)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
```

```
# Build a simple fully connected neural network model
model = Sequential([
    Dense(32, activation='relu', input_shape=(X_train.shape[1],)),
    Dense(16, activation='relu'),
    Dense(1, activation='sigmoid')
])

model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])

# Train the model
history = model.fit(
    X_train, y_train,
    validation_data=(X_test, y_test),
    epochs=10,
    batch_size=128,
    verbose=1
)

# Evaluate the model
y_pred = (model.predict(X_test) > 0.5).astype("int32")
print(classification_report(y_test, y_pred))
print(confusion_matrix(y_test, y_pred))
```

```
/usr/local/lib/python3.13/dist-packages/keras/src/layers/core/dense.py:106: UserWarning: Do not pass an `input_shape`/`input
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Epoch 1/10
1041/1041 ━━━━━━━━━━━━━━━━━━━━ 4s 3ms/step - accuracy: 0.9965 - loss: 0.0279 - val_accuracy: 0.9991 - val_loss: 0.0035
Epoch 2/10
1041/1041 ━━━━━━━━━━━━━━━━━━━━ 3s 2ms/step - accuracy: 0.9991 - loss: 0.0040 - val_accuracy: 0.9991 - val_loss: 0.0032
Epoch 3/10
1041/1041 ━━━━━━━━━━━━━━━━━━━━ 3s 3ms/step - accuracy: 0.9992 - loss: 0.0036 - val_accuracy: 0.9992 - val_loss: 0.0030
Epoch 4/10
1041/1041 ━━━━━━━━━━━━━━━━━━━━ 3s 3ms/step - accuracy: 0.9992 - loss: 0.0032 - val_accuracy: 0.9993 - val_loss: 0.0027
Epoch 5/10
1041/1041 ━━━━━━━━━━━━━━━━━━━━ 5s 3ms/step - accuracy: 0.9993 - loss: 0.0030 - val_accuracy: 0.9994 - val_loss: 0.0028
Epoch 6/10
1041/1041 ━━━━━━━━━━━━━━━━━━━━ 3s 2ms/step - accuracy: 0.9993 - loss: 0.0028 - val_accuracy: 0.9994 - val_loss: 0.0028
Epoch 7/10
1041/1041 ━━━━━━━━━━━━━━━━━━━━ 3s 2ms/step - accuracy: 0.9994 - loss: 0.0025 - val_accuracy: 0.9992 - val_loss: 0.0027
Epoch 8/10
1041/1041 ━━━━━━━━━━━━━━━━━━━━ 3s 3ms/step - accuracy: 0.9994 - loss: 0.0024 - val_accuracy: 0.9995 - val_loss: 0.0026
Epoch 9/10
1041/1041 ━━━━━━━━━━━━━━━━━━━━ 4s 4ms/step - accuracy: 0.9995 - loss: 0.0022 - val_accuracy: 0.9994 - val_loss: 0.0025
Epoch 10/10
1041/1041 ━━━━━━━━━━━━━━━━━━━━ 3s 3ms/step - accuracy: 0.9995 - loss: 0.0022 - val_accuracy: 0.9994 - val_loss: 0.0026
1041/1041 ━━━━━━━━━━━━━━━━━━━━ 1s 1ms/step
              precision    recall  f1-score   support
```

```
          0.0       1.00      1.00      1.00     33230
          1.0       0.92      0.78      0.84        72

    accuracy                           1.00     33302
   macro avg       0.96      0.89      0.92     33302
weighted avg       1.00      1.00      1.00     33302

[[33225     5]
 [   16    56]]
```

```
model = Sequential([
Dense(32, input_dim=X_train.shape[1], activation="relu"),
Dense(16, activation="relu"),
Dense(1, activation="sigmoid")

# Binary classification
])
```

```
/usr/local/lib/python3.13/dist-packages/keras/src/layers/core/dense.py:106: UserWarning: Do not pass an `input_shape`/`input
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

```
model. compile(
optimizer="adam",
loss="binary_crossentropy",
metrics=["accuracy", tf.keras.metrics.Precision(), tf.keras.metrics.Recall()]
)
```

```
# Compute class weights: {0: weight_for_0, 1: weight_for_1}
from sklearn.utils import class_weight
class_weights = class_weight.compute_class_weight(
class_weight="balanced",
classes=np.unique(y_train),
y=y_train
)
class_weights = dict(enumerate(class_weights))
print("Class Weights:", class_weights)
```

```
Class Weights: {0: np.float64(0.501075826994982), 1: np.float64(232.87937062937064)}
```

```
history = model.fit(
X_train, y_train,
validation_split=0.2,
epochs=10,
batch_size=2048,
class_weight=class_weights,
verbose=2

)
```

```
Epoch 1/10
53/53 - 3s - 59ms/step - accuracy: 0.6569 - loss: 0.3857 - precision: 0.0056 - recall: 0.9364 - val_accuracy: 0.9515 - val_l
Epoch 2/10
53/53 - 0s - 9ms/step - accuracy: 0.9759 - loss: 0.2332 - precision: 0.0726 - recall: 0.9045 - val_accuracy: 0.9893 - val_lo
Epoch 3/10
53/53 - 0s - 7ms/step - accuracy: 0.9892 - loss: 0.1724 - precision: 0.1506 - recall: 0.9091 - val_accuracy: 0.9865 - val_lo
Epoch 4/10
53/53 - 0s - 6ms/step - accuracy: 0.9888 - loss: 0.1446 - precision: 0.1458 - recall: 0.9091 - val_accuracy: 0.9879 - val_lo
Epoch 5/10
53/53 - 0s - 6ms/step - accuracy: 0.9876 - loss: 0.1279 - precision: 0.1345 - recall: 0.9182 - val_accuracy: 0.9857 - val_lo
Epoch 6/10
53/53 - 0s - 6ms/step - accuracy: 0.9885 - loss: 0.1150 - precision: 0.1441 - recall: 0.9273 - val_accuracy: 0.9866 - val_lo
Epoch 7/10
53/53 - 0s - 6ms/step - accuracy: 0.9877 - loss: 0.1032 - precision: 0.1357 - recall: 0.9273 - val_accuracy: 0.9851 - val_lo
Epoch 8/10
53/53 - 0s - 6ms/step - accuracy: 0.9855 - loss: 0.0942 - precision: 0.1187 - recall: 0.9364 - val_accuracy: 0.9850 - val_lo
Epoch 9/10
53/53 - 0s - 6ms/step - accuracy: 0.9858 - loss: 0.0890 - precision: 0.1211 - recall: 0.9409 - val_accuracy: 0.9848 - val_lo
Epoch 10/10
53/53 - 0s - 7ms/step - accuracy: 0.9862 - loss: 0.0823 - precision: 0.1246 - recall: 0.9455 - val_accuracy: 0.9873 - val_lo
```

```
y_pred =(model.predict(X_test) > 0.5).astype("int32")

print("\nClassification Report:")
print(classification_report(y_test, y_pred, digits=4))

print("Confusion Matrix:")
print(confusion_matrix(y_test, y_pred))
```

```
1041/1041 ━━━━━━━━━━━━━━━━━━━━ 3s 3ms/step

Classification Report:
```

```
                precision    recall  f1-score   support

           0.0     0.9998    0.9884    0.9940     33230
           1.0     0.1438    0.9028    0.2481        72

    accuracy                         0.9882     33302
   macro avg     0.5718    0.9456    0.6211     33302
weighted avg     0.9979    0.9882    0.9924     33302

Confusion Matrix:
[[32843   387]
 [    7    65]]
```

Start coding or generate with AI.